

Gradient Descent for Logistic Regression

The objective is to find the parameters $\vec{w}$ and b that minimize the logistic regression cost function, $J(\vec{w}, b)$. This is achieved using the **gradient descent** algorithm.

The Gradient Descent Algorithm

The algorithm involves repeatedly updating the parameters $\vec{w}$ and b until convergence.

- **Update Rule:**

- $w_j := w_j - \alpha \frac{\partial J(\vec{w}, b)}{\partial w_j}$

- $b := b - \alpha \frac{\partial J(\vec{w}, b)}{\partial b}$

- **Derivatives (The "Gradient" Part):**

- The partial derivative with respect to a weight w_j is:

- $$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}$$

- The partial derivative with respect to the bias b is:

- $$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

- **Important Note on Execution:** These updates must be **simultaneous**. Calculate the new values for all w_j and b *before* updating any of them.
-

Comparison with Linear Regression 🤖

The update rules for gradient descent in logistic regression appear **identical** to those for linear regression. However, they are **not the same algorithm**.

- The key difference lies in the definition of the model's prediction, $f_{\vec{w}, b}(\vec{x})$:
 - **Linear Regression:** $f_{\vec{w}, b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$
 - **Logistic Regression:** $f_{\vec{w}, b}(\vec{x}) = \frac{1}{1 + e^{-(\vec{w} \cdot \vec{x} + b)}}$ (the sigmoid function)

This fundamental difference in the function f means the calculations for the error term $(f(\vec{x}) - y)$ and the resulting gradients will be completely different, leading to two distinct algorithms despite the similar-looking update formulas.

Practical Implementation Tips 💡

- **Monitoring Convergence:** Just as with linear regression, you can plot the cost function $J(\vec{w}, b)$ over the number of iterations to ensure that it is consistently decreasing and gradient descent is working correctly.

- **Vectorization:** For computational efficiency, the loops in gradient descent can be replaced with vectorized operations, significantly speeding up the training process.
- **Feature Scaling:** Scaling features to a similar range (e.g., -1 to 1) can help gradient descent converge much faster. The same techniques used for linear regression apply here.